

Google Agent Development Kit (ADK)

Building a Multi-Agent Campus Assistant, Layer by Layer

Agenda

1

What is ADK?

Google's open-source, model-agnostic framework

2

Building Agents with LlmAgent

Agents, tools, and orchestration basics

3

Model Flexibility via LiteLLM

Swap LLM providers without rewriting agents

4

Native MCP Tool Integration

Connect agents to MCP servers and build custom tools

5

Agent Garden

Pre-built agent patterns and templates

6

Streaming

Bidirectional audio and video with agents

7

Hierarchical Structures & Routing

Multi-agent orchestration in ADK

8

Agent-to-Agent Delegation

Handing off tasks between agents

9

Error Handling & Debugging

Keeping multi-agent systems reliable

SECTION 1

What is ADK?

Google's toolkit for building, testing, and deploying AI agents.

01

Open-source and model-agnostic by design

02

LlmAgent: the core building block for agents, tools, and orchestration

03

Model flexibility across providers via LiteLLM

An Open-Source, Model-Agnostic Framework

Key Ideas

- Built by Google, released as open source — free to use and inspect
- “Model-agnostic”: not locked to Gemini — works with many LLM providers
- Same code path for a simple chatbot or a full multi-agent system
- Today's project: a Campus Assistant for SMIT students, built one layer at a time

```
campus_assistant.py - v1

from google.adk.agents import Agent

root_agent = Agent(
    name="campus_assistant",
    model="gemini-2.0-flash",
    instruction=(
        "Friendly SMIT campus assistant."
    ),
)
```

LlmAgent, Tools, and Orchestration

Key Ideas

- LlmAgent = an Agent that reasons with an LLM and can call tools
- A tool is just a normal Python function with a clear docstring
- ADK decides when to call a tool based on the user's question
- Orchestration: the agent plans, calls tools, then answers

campus_assistant.py - v2

```
from google.adk.agents import LlmAgent

def check_class_schedule(day: str) -> dict:
    classes = {"Monday": "Python Basics, 6 PM"}
    return {"status": "success",
            "class": classes.get(day, "None")}

root_agent = LlmAgent(
    name="campus_assistant", model="gemini-2.0-flash",
    tools=[check_class_schedule],
)
```

Model Flexibility via LiteLLM

Key Ideas

- LiteLLM lets one agent call almost any model provider
- Swap Gemini for OpenAI, Anthropic, or a local model — same agent code
- Useful for cost, speed, or privacy comparisons during development
- Only the model= line changes; tools and instructions stay the same

```
campus_assistant.py - v3

from google.adk.models.lite_llm import LiteLlm

root_agent = LlmAgent(
    name="campus_assistant",
    model=LiteLlm(model="openai/gpt-4o-mini"),
    tools=[check_class_schedule],
)
# Same agent, a different LLM provider
```

SECTION 2

MCP Tools & Agent Garden

Connecting agents to the outside world and reusing proven patterns.

01

Native MCP support: plug in any MCP-compatible tool server

02

Agent Garden: ready-made agent patterns and templates

03

Bidirectional audio and video streaming for live conversations

Native MCP Tool Integration

✗ Without MCP

Custom glue code for every external tool or API — rewritten per agent, per project.

✓ With MCP

One protocol connects any MCP server — filesystem, database, or custom tool — with a few lines.

`campus_assistant.py` — v4

```
from google.adk.tools.mcp_tool import (
    MCPToolset, StdioServerParameters)

docs_tools = MCPToolset(
    connection_params=StdioServerParameters(
        command="npx",
        args=["-y", "mcp-server-filesystem",
              "./docs"],
    ))

root_agent = LlmAgent(
    name="campus_assistant", model="gemini-2.0-flash",
    tools=[check_class_schedule, docs_tools],
)
```

Agent Garden: Pre-Built Patterns

Key Ideas

- A library of ready-made agent patterns and templates
- Covers common needs: RAG Q&A, research agents, data-analysis agents
- Start from a template and adapt it instead of building from zero
- Great for prototyping before writing a fully custom agent

`campus_assistant.py - v5`

```
# Agent Garden ships a ready-made RAG pattern
from google.adk.samples.rag_qa import build_rag_agent

docs_qa_agent = build_rag_agent(
    corpus_path="./docs",
    name="docs_qa_agent",
)
# Adapt the template instead of writing RAG by hand
```

Bidirectional Audio & Video Streaming

Key Ideas

- ADK supports live, two-way audio and video with an agent
- “Bidirectional”: the agent can listen and speak at the same time
- Same agent and tools — just a different run mode
- Useful for a campus kiosk students can talk to directly

`campus_assistant.py - v6`

```
from google.adk.agents.run_config import (
    RunConfig, StreamingMode)

root_agent = LlmAgent(
    name="campus_assistant",
    model="gemini-2.0-flash-live",
    tools=[check_class_schedule],
)

run_config = RunConfig(
    streaming_mode=StreamingMode.BIDI)
```

Multi-Agent Orchestration

Composing several focused agents into one reliable system.

01

Hierarchical structures: a router agent directs traffic

02

Agent-to-agent delegation hands off the full conversation

03

Error handling and debugging keep the system trustworthy

Hierarchical Structures and Routing

Key Ideas

- A root agent can hold sub_agents instead of only tools
- The root agent reads the question and routes it to the right sub-agent
- Each sub-agent stays small and focused on one job
- This is how ADK scales beyond a single agent

`campus_assistant.py - v7`

```
schedule_agent = LlmAgent(  
    name="schedule_agent", model="gemini-2.0-flash",  
    tools=[check_class_schedule])  
  
docs_agent = LlmAgent(  
    name="docs_agent", model="gemini-2.0-flash",  
    tools=[docs_tools])  
  
root_agent = LlmAgent(  
    name="campus_router", model="gemini-2.0-flash",  
    instruction="Route schedule vs docs questions.",  
    sub_agents=[schedule_agent, docs_agent],  
)
```

Agent-to-Agent Delegation

Key Ideas

- Delegation hands off the whole turn, not just an answer
- ADK's built-in transfer tool moves the conversation between agents
- The receiving agent gets the conversation state automatically
- No manual copying of chat history between agents

`campus_assistant.py - v8`

```
# Inside schedule_agent's instruction:
"""
If the question is about course content,
call transfer_to_agent("docs_agent").
"""

# ADK performs the handoff and carries the
# conversation state - no manual state passing
```

Error Handling and Debugging

✗ Unhandled

A missing key or failed call crashes the tool and stalls the whole conversation.

✓ Handled

Catch the error, return a clear status, and let the agent keep talking to the student.

campus_assistant.py - v9

```
def check_class_schedule(day: str) -> dict:
    try:
        return {"status": "success",
                "class": classes[day]}
    except KeyError:
        return {"status": "error",
                "message": f"No class on {day}"}

# Debug with: adk run --verbose
```

All Pieces Assembled: Your Campus Assistant

What We Layered In

- A router agent (campus_router)
- Two focused sub-agents: schedule_agent, docs_agent
- An MCP-connected tool for course docs
- A LiteLLM-ready model slot
- Try/except error handling in every tool
- Every layer maps to a subtopic we just covered

```
campus_assistant.py — final

def check_class_schedule(day: str) -> dict:
    try:
        return {"status": "success", "class": classes[day]}
    except KeyError:
        return {"status": "error", "message": f"No class on {day}"}

docs_tools = MCPToolset(connection_params=StdioServerParameters(
    command="npx", args=["-y", "mcp-server-filesystem", "./docs"]))

schedule_agent = LlmAgent(name="schedule_agent",
    model="gemini-2.0-flash", tools=[check_class_schedule])

docs_agent = LlmAgent(name="docs_agent",
    model="gemini-2.0-flash", tools=[docs_tools])

root_agent = LlmAgent(
    name="campus_router",
    model=LiteLlm(model="openai/gpt-4o-mini"), # swappable
    instruction="Route schedule vs docs questions.",
    sub_agents=[schedule_agent, docs_agent],
)
```

Module Complete — What You Now Know:

- ✓ What ADK is and why it's open-source and model-agnostic
- ✓ How bidirectional audio and video streaming works in ADK
- ✓ How to build an agent with LlmAgent, tools, and orchestration
- ✓ How hierarchical structures route work across sub-agents
- ✓ How LiteLLM makes an agent's model provider swappable
- ✓ How agent-to-agent delegation hands off a full conversation
- ✓ How native MCP support connects agents to external tools
- ✓ How to add error handling and debug a multi-agent system
- ✓ How to build a custom tool and when to reach for Agent Garden
- ✓ How every piece assembles into one working Campus Assistant